

Índice

1	Resumen Ejecutivo	2
2	Arquitectura del Sistema	3
3	Smart Contract: Diseño e Implementación	4
3.1	Optimizaciones de gas	4
3.2	Estructuras y variables de estado	4
3.3	Modificadores de seguridad	4
3.4	Sistema de Padrón Electoral (Whitelist)	5
4	Frontend: Desarrollo Angular	6
4.1	Componente principal: VotacionComponent	6
4.2	Servicio BlockchainService	6
4.3	Validaciones proactivas	6
5	Despliegue en Sepolia	8
5.1	Compilación del contrato	8
5.2	Despliegue exitoso	8
5.3	Verificación de actualización del frontend	9
6	Pruebas End-to-End	11
6.1	Interfaz inicial del Panel de Control Operativo	11
6.2	Registro de votantes con polling on-chain	11
6.3	Validación del banner de autorización para votante autorizado	12
6.4	Control de acceso del administrador	13
6.5	Estado tras flujo completo de votación	14
7	Bugs identificados y resueltos	16
7.1	ABI desactualizado tras redespliegue del contrato	16
7.2	Race condition en tx.wait() de ethers.js v6	16
7.3	Bug de NgZone / Change Detection en Angular	17
7.4	UX de mensajes superpuestos	17
8	Despliegue en Firebase Hosting	18
8.1	Justificación arquitectónica	18
8.2	Instalación de herramientas	18
8.3	Compilación de producción de Angular	18
8.4	Inicialización y configuración de Firebase	19
8.5	Pipeline de Integración Continua con GitHub Actions	19
8.6	Bug identificado durante el despliegue inicial	20
8.7	Despliegue exitoso	20
9	Cumplimiento del Documento de Visión	22

10 Trabajo Futuro	23
10.1 Carga masiva del padrón electoral mediante CSV	23
10.2 Auto-registro vinculado a identidad universitaria	23
10.3 Soulbound Tokens (SBT) como credencial estudiantil	23
10.4 Privacidad mediante Zero-Knowledge Proofs	23
10.5 Despliegue en mainnet o L2	23
11 Conclusiones	24

1 Resumen Ejecutivo

El presente documento describe la implementación técnica del proyecto **Voto Universitario**, una aplicación descentralizada (DApp) desarrollada para satisfacer los requerimientos establecidos en el Documento de Visión del proyecto. El sistema fue construido sobre la red de pruebas Ethereum Sepolia y permite gestionar procesos de votación estudiantiles con garantías criptográficas de seguridad, transparencia e inmutabilidad.

Durante el desarrollo se implementaron y validaron las siguientes funcionalidades clave: registro de candidatos, control de fases electorales (apertura y cierre), gestión de padrón electoral con whitelist on-chain, emisión de votos firmada mediante MetaMask, validación criptográfica de voto único, y consulta de resultados en tiempo real. Adicionalmente se realizaron mejoras significativas en la experiencia de usuario, incluyendo un sistema de notificaciones temporales y diagnóstico avanzado mediante polling al contrato.

Datos clave del despliegue

Concepto	Valor
Red Blockchain	Ethereum Sepolia Testnet
Dirección del Contrato	0x7E94C235B9E6f16A3A0b99724CE69Cfb3ef9786b
Cuenta Administradora	0xE60E4F42913DE6e4b0975D146d268f4A34064549
URL Pública de la DApp	https://voto-universitario-770b7.web.app
Hosting	Firebase Hosting + CI/CD con GitHub Actions
Compilador Solidity	v0.8.24 (EVM target: paris)
Framework Frontend	Angular 21 (Standalone Components)
Librería Web3	ethers.js v6
Entorno de Desarrollo	Hardhat 2.22.17

2 Arquitectura del Sistema

La arquitectura del proyecto sigue el modelo estándar de aplicaciones descentralizadas, separando claramente la capa de presentación, la lógica de negocio inmutable on-chain y la capa de autenticación criptográfica del usuario.

2.1 Componentes principales

- **Smart Contract** (Capa de lógica y datos): Desarrollado en Solidity 0.8.24 y desplegado en la red Sepolia. Contiene toda la lógica de votación, los mapeos de candidatos, votantes autorizados y registros de votos. Es inmutable una vez desplegado.
- **Frontend** (Capa de presentación): Aplicación Angular 21 que actúa como interfaz gráfica entre el usuario y el contrato. Se comunica con la blockchain a través de la librería ethers.js v6.
- **MetaMask** (Capa de autenticación): Wallet del navegador que custodia las llaves privadas del usuario y firma cada transacción enviada a la blockchain.
- **Red Sepolia** (Infraestructura): Red de pruebas pública de Ethereum donde reside el contrato y se ejecutan todas las transacciones.

2.2 Flujo de comunicación

Cuando un usuario realiza una acción sensible (registrar candidato, emitir voto, autorizar votante), el flujo es el siguiente:

- El frontend invoca al servicio Angular `BlockchainService`.
- `BlockchainService` construye la transacción mediante ethers.js.
- MetaMask presenta la transacción al usuario para firma.
- La transacción firmada se transmite a la red Sepolia.
- Los mineros incluyen la transacción en un bloque y la propagan.
- El frontend ejecuta polling periódico al contrato para detectar el cambio de estado y refrescar la interfaz.

3 Smart Contract: Diseño e Implementación

3.1 Optimizaciones de gas

El contrato fue diseñado priorizando la eficiencia en consumo de gas. Se aplicaron las siguientes optimizaciones técnicas:

- Uso de `bytes32` en lugar de `string` para nombres de candidatos. Los strings requieren slots dinámicos de almacenamiento y operaciones costosas; `bytes32` ocupa un único slot fijo de 32 bytes y reduce significativamente el gas de escritura.
- Uso de `uint32` en lugar de `uint256` para identificadores y contadores de votos. `uint32` soporta hasta 4,294,967,295 valores, suficiente para cualquier elección universitaria realista, y permite empaquetar múltiples variables en un mismo slot.
- Función de lectura batch `obtenerTodosLosCandidatos()` que retorna todos los candidatos en una sola llamada RPC, reduciendo la latencia percibida en el frontend.

3.2 Estructuras y variables de estado

```
struct Candidato {
    uint32 id;
    bytes32 nombre;
    uint32 votos;
}

address public administrador;
bool public eleccionAbierta;
uint32 public totalCandidatos;
uint32 public totalVotantesAutorizados;

mapping(uint32 => Candidato) public candidatos;
mapping(address => bool) public yaVoto;
mapping(address => bool) public votanteAutorizado;
```

3.3 Modificadores de seguridad

El contrato implementa tres modificadores que aplican control de acceso a nivel criptográfico:

Modificador	Propósito
<code>soloAdministrador</code>	Restringe la función para que únicamente la cuenta administradora (definida en el constructor) pueda ejecutarla. Aplica a <code>registrarCandidato</code> , <code>registrarVotante</code> , <code>abrirEleccion</code> y <code>cerrarEleccion</code> .
<code>eleccionActiva</code>	Garantiza que la función solo se ejecute cuando el flag <code>eleccionAbierta</code> esté en <code>true</code> . Aplica a la función <code>votar</code> .
<code>soloVotanteAutorizado</code>	Verifica que la dirección emisora esté presente en el padrón electoral (mapping <code>votanteAutorizado</code>). Aplica a la función <code>votar</code> .

3.4 Sistema de Padrón Electoral (Whitelist)

Para cumplir con el requisito del Documento de Visión de *registrar votantes autorizados*, se implementó un sistema de whitelist on-chain. Solo las direcciones explícitamente autorizadas por el administrador pueden emitir votos.

El contrato ofrece dos métodos de registro de votantes:

- `registrarVotante(address)`: Registra una sola dirección. Adecuado para padrones pequeños o correcciones individuales.
- `registrarVotantesEnLote(address[])`: Registra múltiples direcciones en una sola transacción, optimizando el gas en escenarios de padrón masivo. Esta función no se expone actualmente en la interfaz gráfica pero queda disponible en el contrato como prueba de capacidad técnica para escalar a miles de votantes.

Adicionalmente, la función `abrirEleccion` incluye una validación de seguridad que requiere al menos un votante autorizado y al menos un candidato registrado antes de permitir la apertura del proceso electoral, evitando elecciones inválidas.

4 Frontend: Desarrollo Angular

4.1 Componente principal: VotacionComponent

La interfaz fue construida como un componente standalone de Angular 21, ubicado en `frontend/src/app/components/votacion`. Maneja el estado completo de la aplicación, incluyendo:

- Conexión MetaMask y dirección de la cuenta activa
- Lista de candidatos con conteo de votos en tiempo real
- Estado del padrón electoral (total y autorización del usuario actual)
- Estado de la elección (abierta o cerrada)
- Mensajes de notificación con auto-dismiss temporal

4.2 Servicio BlockchainService

Toda la comunicación con el contrato se centraliza en el servicio `BlockchainService`, que envuelve las llamadas de ethers.js v6 con tres helpers críticos:

conTimeout<T>

Aplica un timeout configurable a cualquier promesa de ethers.js. Previene que la interfaz se congele indefinidamente si MetaMask o el provider tardan en responder. Implementado mediante `Promise.race` entre la operación original y un timeout.

esperarCondicion

Ejecuta polling periódico al contrato (cada 2 segundos, timeout total de 60 segundos) hasta detectar que un predicado se cumple. Esto resuelve un problema conocido del `tx.wait()` de ethers v6: en ocasiones la confirmación WebSocket entre MetaMask y el provider no se sincroniza correctamente aunque la transacción ya esté minada en la blockchain.

enviarTx

Wrapper que combina `conTimeout` y maneja el caso particular en que MetaMask demora en confirmar el envío. Si se agota el timeout pero la transacción ya pudo haber sido enviada, continúa al polling como verificación final.

4.3 Validaciones proactivas

Para optimizar la experiencia de usuario y evitar gastos de gas innecesarios, el frontend ejecuta validaciones de lectura (no consumen gas) antes de solicitar firma a MetaMask:

- Antes de votar, verifica `esVotanteAutorizado(cuenta)` y `yaVoto(cuenta)`. Si alguna falla, muestra un mensaje claro al usuario sin abrir MetaMask.

- Antes de registrar un votante, valida formato de la dirección Ethereum (longitud 42, prefijo 0x).
- Antes de abrir la elección, verifica que existan candidatos y votantes en el padrón.

Esta estrategia, combinada con las validaciones criptográficas del contrato (**require**), produce una arquitectura de seguridad de doble capa: el frontend mejora la usabilidad pero el contrato siempre tiene la palabra final.

5 Despliegue en Sepolia

5.1 Compilación del contrato

El contrato fue compilado utilizando Hardhat con target EVM *paris*. La compilación se ejecutó exitosamente tras la limpieza previa de caché con `npm hardhat clean`.

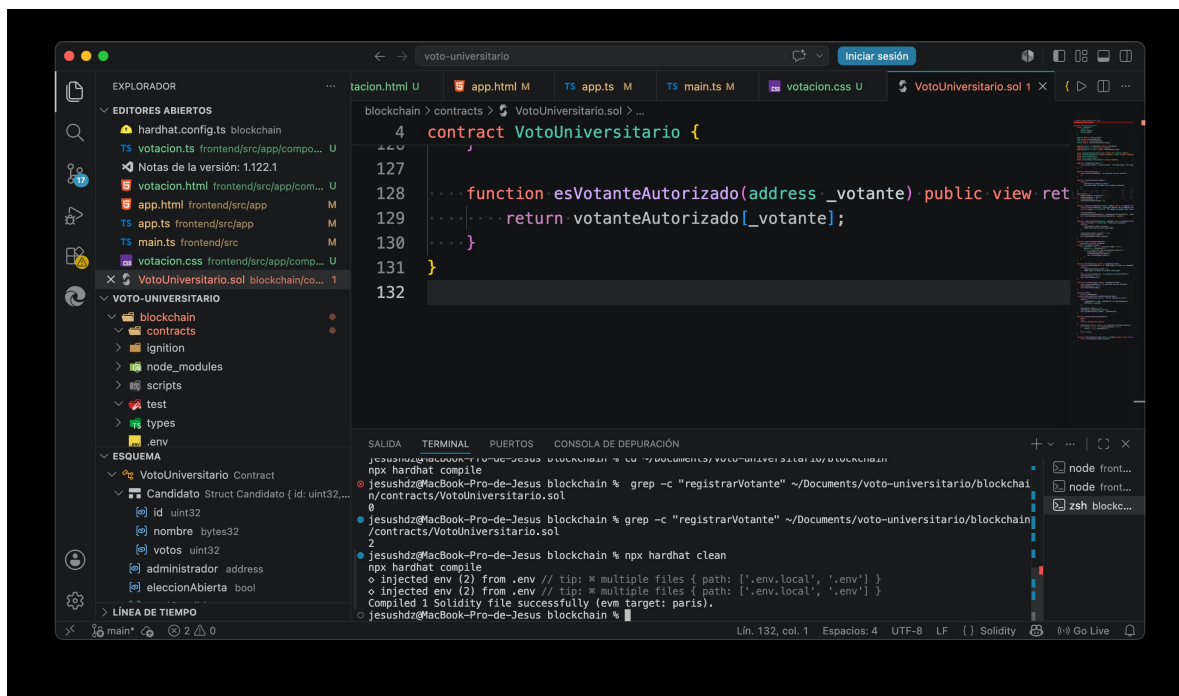


Figura 1. Compilación exitosa del contrato *VotoUniversitario.sol* con Hardhat

5.2 Despliegue exitoso

El despliegue se realizó mediante el script personalizado `scripts/deploy.ts` que utiliza un signer importado desde las variables de entorno (`.env`). El contrato fue minado en la red Sepolia en aproximadamente 30 segundos.

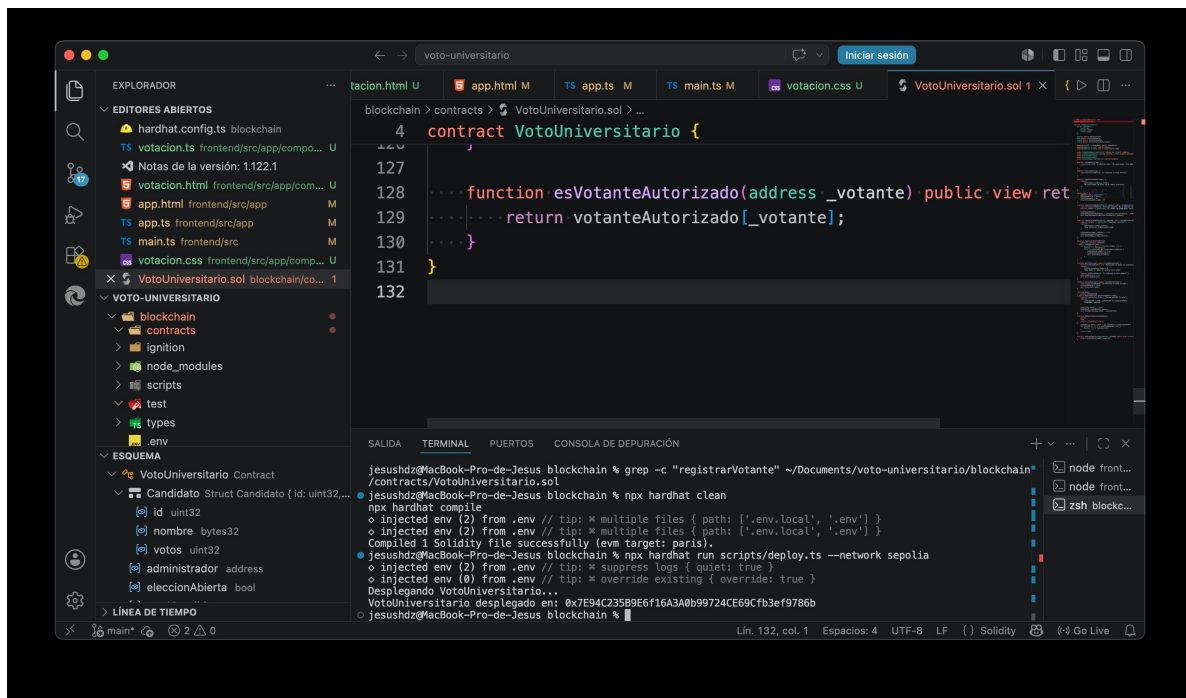


Figura 2. Despliegue del contrato en Sepolia. Dirección obtenida: `0x7E94C235B9E6f16A3A0b99724CE69Cfb3ef9786b`

5.3 Verificación de actualización del frontend

Después del despliegue, se actualizó la dirección del contrato y el ABI en el frontend. La actualización fue verificada mediante `grep` para confirmar que ambos archivos (el ABI en `assets` y el servicio TypeScript) contienen las nuevas funciones de padrón electoral.

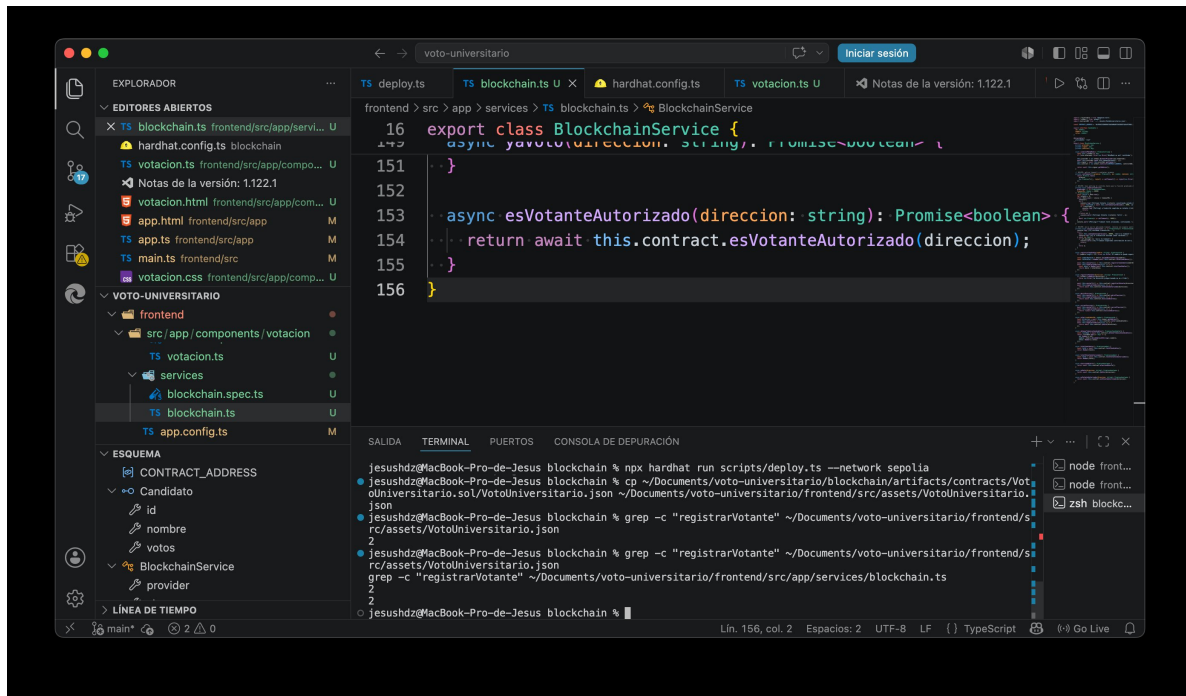


Figura 3. Verificación grep confirma la presencia de la función *registrarVotante* tanto en el archivo ABI (*VotoUniversitario.json*) como en el servicio Angular (*blockchain.ts*)

6 Pruebas End-to-End

Se ejecutó un flujo de prueba completo desde el frontend desplegado en localhost, contra el contrato vivo en la red Sepolia. Las siguientes capturas documentan los hitos del flujo.

6.1 Interfaz inicial del Panel de Control Operativo

Tras conectar MetaMask con la cuenta administradora, se carga la interfaz principal con las tres secciones operativas: Registrar Nuevo Candidato, Padrón Electoral y Fases de la Elección. El estado inicial del contrato es CERRADA / PAUSADA, sin candidatos ni votantes registrados.

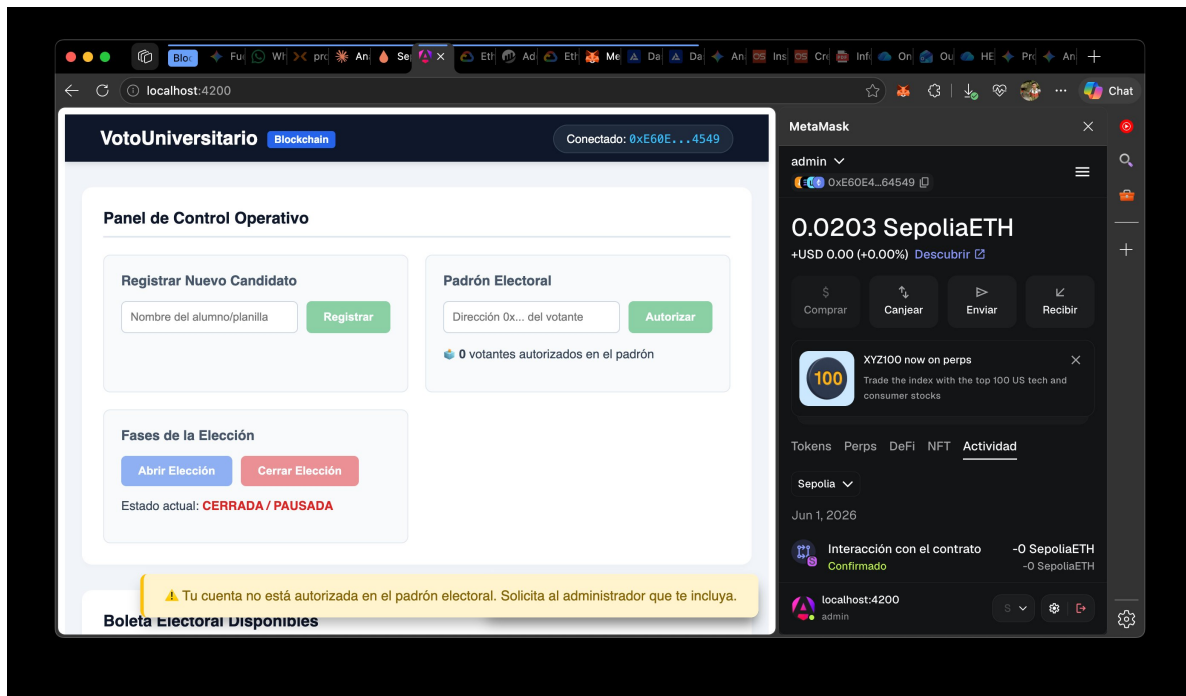


Figura 4. Panel de Control Operativo en su estado inicial, listo para registrar candidatos y votantes

6.2 Registro de votantes con polling on-chain

El registro de votantes en el padrón fue verificado tanto en la interfaz como en los logs de la consola del navegador. Se confirma que el polling al contrato detecta correctamente el cambio de estado tras la transacción.

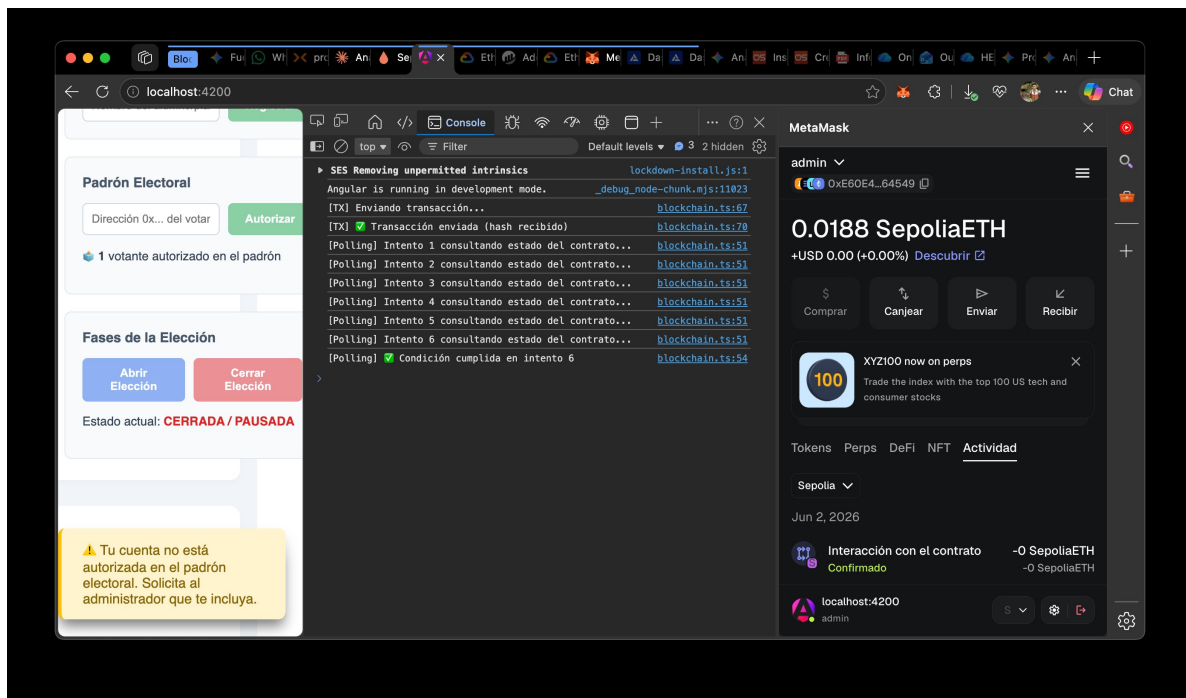


Figura 5. *Logs del polling muestran la secuencia [TX] Enviando transacción – Transacción enviada – 6 intentos de polling – Condición cumplida. El contador del padrón sube de 0 a 1*

6.3 Validación del banner de autorización para votante autorizado

Una cuenta diferente a la administradora (alumno1) se conecta a la DApp. Como esta cuenta fue previamente añadida al padrón, el banner verde de confirmación aparece automáticamente.

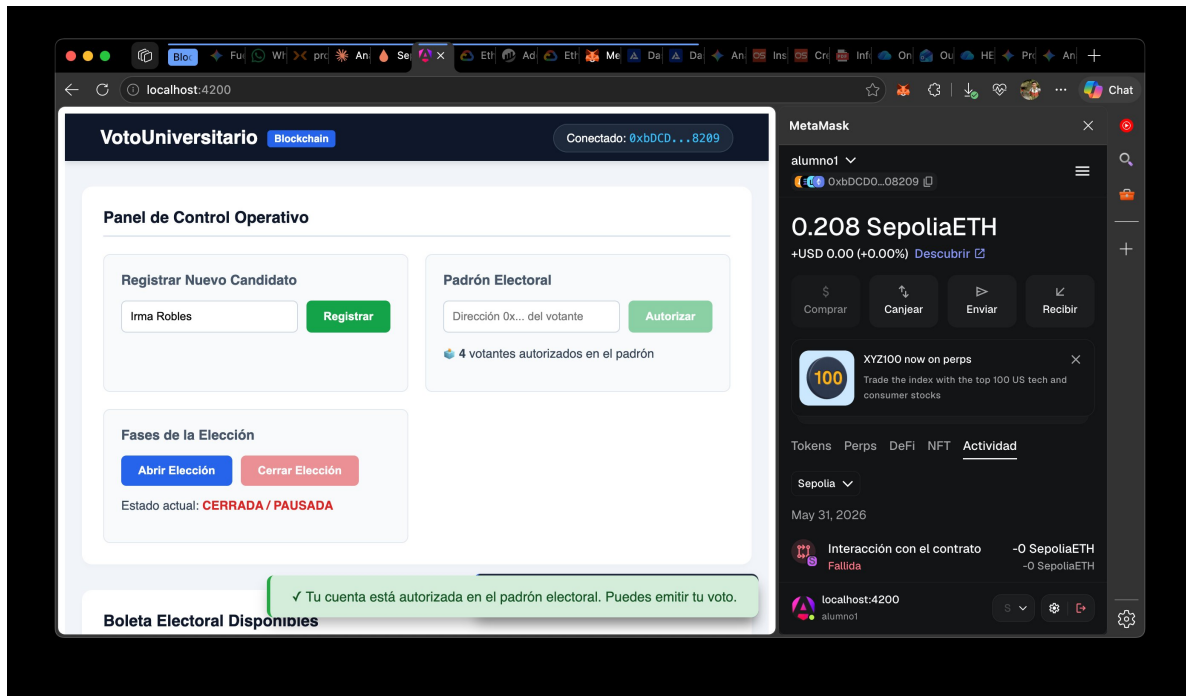


Figura 6. La cuenta *alumno1* conectada con saldo *0.208 SepoliaETH* visualiza el banner de autorización en color verde. El padrón muestra *4* votantes autorizados

6.4 Control de acceso del administrador

Cuando una cuenta no administradora intenta ejecutar funciones reservadas (como registrar candidatos), el modifier `soloAdministrador` rechaza la transacción y el frontend muestra un mensaje de error claro en lugar de proceder.

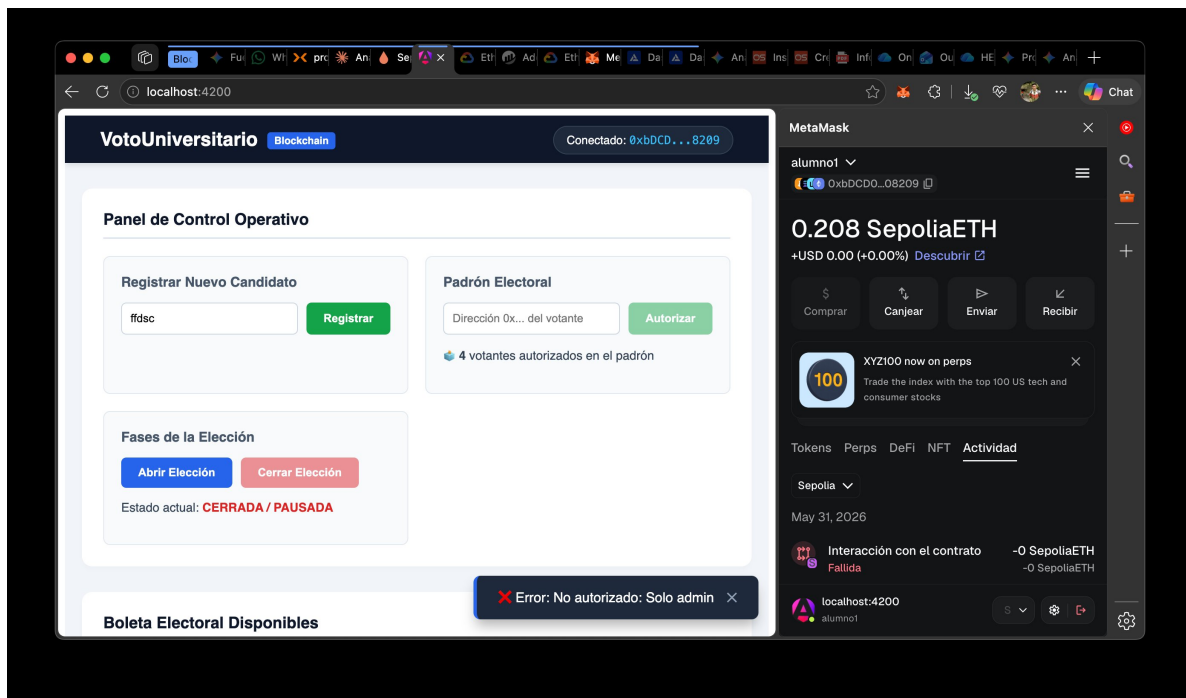


Figura 7. Intento fallido de registrar un candidato desde una cuenta no administradora. El contrato rechaza la operación con el mensaje No autorizado: Solo admin

6.5 Estado tras flujo completo de votación

La siguiente captura corresponde a una versión previa de prueba del sistema, donde se demostró el flujo completo: registro de candidatos, apertura de elección, emisión de dos votos válidos (resultando en 2 votos para Jorge Martinez), y cierre del proceso electoral con resultados inmutables on-chain.

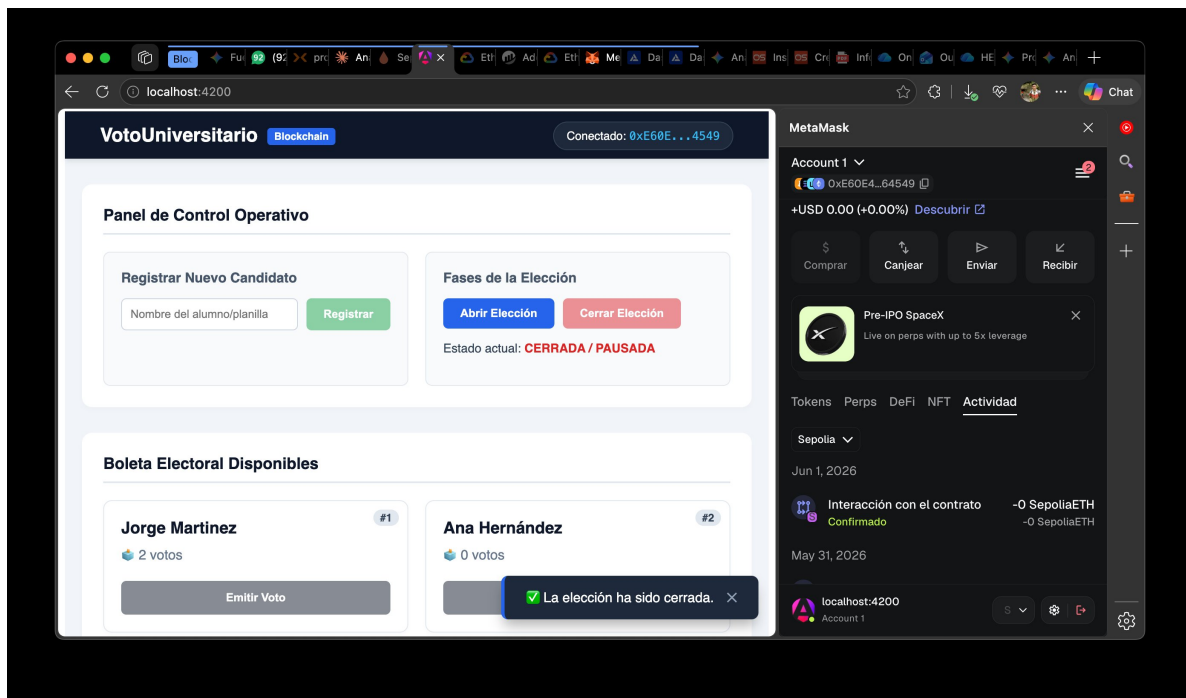


Figura 8. Elección cerrada con resultados consolidados. Jorge Martinez obtuvo 2 votos, Ana Hernández 0. Los botones de Emitir Voto aparecen deshabilitados porque la elección está pausada

7 Bugs identificados y resueltos

Durante el desarrollo se identificaron y resolvieron varios bugs técnicos que ilustran retos comunes en el desarrollo de aplicaciones descentralizadas. A continuación se documenta cada uno junto con la solución implementada.

7.1 ABI desactualizado tras redespiegue del contrato

Síntoma: el frontend lanzaba errores de tipo `CALL_EXCEPTION` al intentar invocar funciones nuevas del contrato. La causa raíz era que el archivo `VotoUniversitario.json` en `frontend/src/assets` contenía la interfaz binaria del contrato anterior.

Solución: tras cada despliegue, se ejecuta el comando `cp` para sobrescribir el ABI del frontend con el más reciente generado por Hardhat en `artifacts/contracts/VotoUniversitario.sol/Vo`. Adicionalmente, se actualiza manualmente la constante `CONTRACT_ADDRESS` en el servicio `BlockchainService`.

7.2 Race condition en `tx.wait()` de ethers.js v6

Síntoma: tras una transacción exitosa, la UI permanecía con el spinner cargando indefinidamente. MetaMask mostraba la transacción como *Confirmado* pero la promesa de `tx.wait()` nunca se resolvía. El usuario debía recargar manualmente la página para ver el cambio reflejado.

Solución: se reemplazó `tx.wait()` por un sistema de polling personalizado (`esperarCondicion`) que consulta directamente el estado del contrato cada 2 segundos hasta detectar el cambio esperado. Esto desacopla la interfaz de la sincronización WebSocket entre MetaMask y el provider.

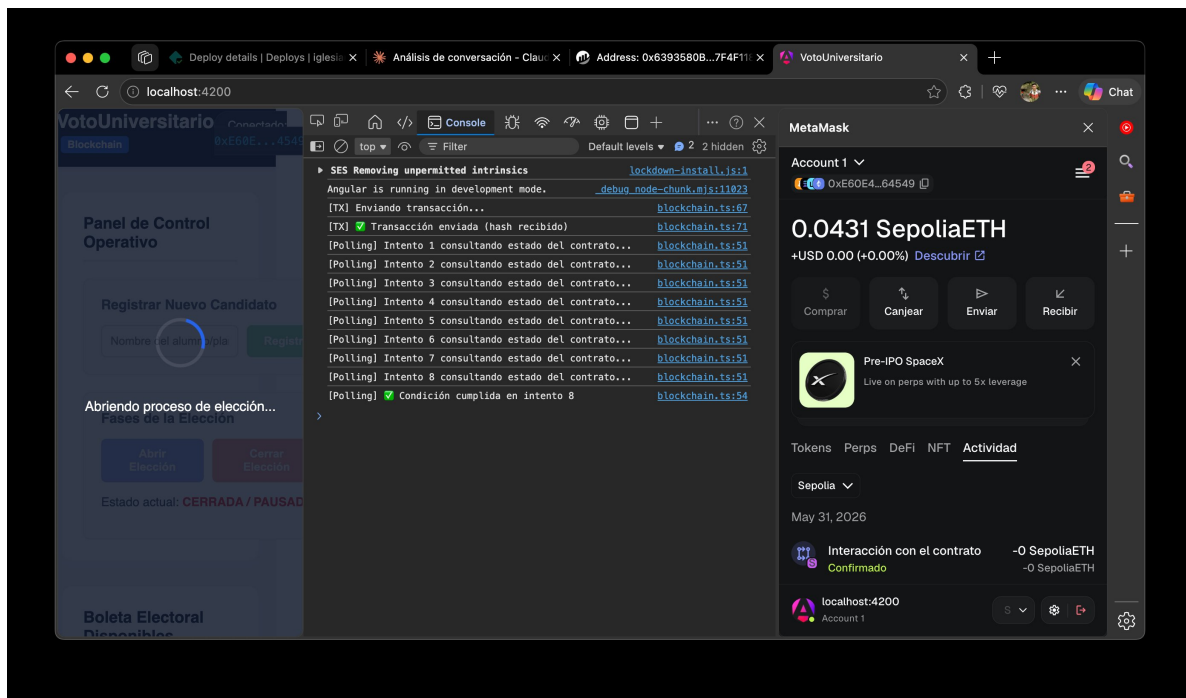


Figura 9. Logs de la consola demostrando el comportamiento del nuevo sistema de polling, que detecta el cambio de estado tras 8 intentos consecutivos

7.3 Bug de NgZone / Change Detection en Angular

Síntoma: incluso con el polling funcionando correctamente, la UI no se actualizaba automáticamente. Los logs mostraban *Condición cumplida* pero el spinner seguía visible y los datos en pantalla no se refrescaban.

Causa raíz: las callbacks asíncronas de ethers.js se ejecutan fuera de la zona de detección de cambios de Angular (NgZone). Cuando una variable cambiaba dentro de estas callbacks, Angular no detectaba el cambio.

Solución: se inyectó `ChangeDetectorRef` en el componente y se llama explícitamente a `this.cdr.detectChanges()` en el bloque `finally` de cada operación asíncrona, forzando manualmente el ciclo de detección de cambios.

7.4 UX de mensajes superpuestos

Síntoma: las notificaciones se acumulaban visualmente cuando ocurrían varias acciones en secuencia, dificultando la lectura del estado real del sistema. Adicionalmente, el banner de autorización permanecía visible indefinidamente, sumándose a los toasts de error y generando información contradictoria.

Solución: se implementó un sistema de auto-dismiss temporal con `setMensajeTemporal` y `mostrarBannerAutorizacionTemporal`, ambos con timers configurables (5s para éxito/info, 7s para errores). El banner de autorización solo se dispara explícitamente al conectar MetaMask, no en cada recarga de datos, evitando la superposición con mensajes de error.

8 Despliegue en Firebase Hosting

Como paso final de la implementación, la aplicación frontend fue publicada en internet utilizando Firebase Hosting de Google. Este servicio ofrece HTTPS automático (requisito indispensable para que MetaMask funcione fuera de localhost), CDN global y un plan gratuito con límites suficientes para un proyecto académico.

Es importante señalar que únicamente el frontend Angular fue publicado en Firebase. El smart contract continúa viviendo en la red Sepolia (dirección `0x7E94C235B9E6f16A3A0b99724CE69Cfb3ef9`) y la wallet del usuario (MetaMask) sigue ejecutándose en el navegador del cliente. Esta separación de capas es justamente lo que define a una aplicación Web3 verdaderamente descentralizada.

8.1 Justificación arquitectónica

Aunque Firebase Hosting podría considerarse un punto de centralización, en realidad solo aloja archivos estáticos (HTML, CSS, JavaScript) que cualquier persona podría replicar en otro proveedor (IPFS, Netlify, Vercel, AWS, etc.). La lógica de negocio y los datos del proceso electoral viven exclusivamente en blockchain. Esto significa que aunque Firebase dejara de operar, el contrato seguiría funcionando indefinidamente y bastaría con desplegar el frontend en otro hosting para restaurar el acceso a la DApp.

8.2 Instalación de herramientas

El primer paso fue instalar globalmente la interfaz de línea de comandos de Firebase, que permite gestionar proyectos y desplegar contenido desde la terminal:

```
npm install -g firebase-tools
```

Posteriormente se autenticó la sesión con la cuenta de Google asociada al proyecto, lo cual abre una ventana del navegador para confirmar permisos:

```
firebase login
```

8.3 Compilación de producción de Angular

Antes del despliegue, se generó una versión optimizada del frontend mediante el compilador de Angular. Este proceso transforma todo el código TypeScript en archivos JavaScript minificados, optimiza assets y crea un bundle listo para producción:

```
ng build
```

El comando generó los archivos compilados en la ruta `dist/voto-universitario/browser`. Es importante destacar que las versiones modernas de Angular (17 en adelante con Application Builder) añaden un subdirectorio `browser` dentro de `dist`, lo cual fue un detalle clave al

configurar el directorio público en Firebase.

8.4 Inicialización y configuración de Firebase

La inicialización del proyecto se realizó con el comando `firebase init` dentro de la carpeta `frontend`. Durante este proceso interactivo se configuraron las siguientes opciones:

- Función seleccionada: *Hosting (Set up deployments for static web apps)*
- Proyecto Firebase: `voto-universitario-770b7` (creado previamente en la consola web de Firebase)
- Directorio público: `dist/voto-universitario/browser`
- Configuración SPA (Single Page Application): Sí, todas las rutas se reescriben a `/index.html`
- Integración con GitHub Actions: Sí, para activar despliegues automáticos en cada push a la rama `main`

El archivo de configuración resultante `firebase.json` quedó estructurado de la siguiente manera:

```
{
  "hosting": {
    "public": "dist/voto-universitario/browser",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

8.5 Pipeline de Integración Continua con GitHub Actions

Adicionalmente, se configuró un flujo de despliegue continuo (CI/CD) utilizando GitHub Actions. Esto significa que cada vez que se hace un push a la rama `main` del repositorio, GitHub ejecuta automáticamente la compilación de Angular y publica los archivos resultantes en Firebase, sin necesidad de intervención manual.

El archivo generado por la CLI de Firebase requirió un ajuste manual: dado que el proyecto Angular vive dentro de una subcarpeta llamada `frontend` (y no en la raíz del repositorio), fue necesario agregar la directiva `defaults.run.working-directory` para indicar a GitHub Actions dónde ejecutar los comandos.

El archivo `.github/workflows/firebase-hosting-merge.yml` quedó configurado así:

```
name: Deploy to Firebase Hosting on merge
on:
  push:
    branches:
      - main
jobs:
  build_and_deploy:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npx ng build
      - uses: FirebaseExtended/action-hosting-deploy@v0
        with:
          repoToken: ${ secrets.GITHUB_TOKEN }
          firebaseServiceAccount: ${ secrets.FIREBASE_SERVICE_ACCOUNT_VOTO_UNIVERSITARIO_770B7 }
          channelId: live
          projectId: voto-universitario-770b7
```

8.6 Bug identificado durante el despliegue inicial

El primer intento de despliegue arrojó una página de error 404 (Page Not Found) al visitar la URL pública. La causa fue que el archivo `firebase.json` apuntaba originalmente a `dist/voto-universitario` sin incluir el subdirectorio `/browser` que Angular 17+ genera automáticamente. Firebase estaba subiendo una carpeta sin un archivo `index.html` válido.

La solución consistió en verificar la estructura real generada por Angular mediante el comando `ls dist/voto-universitario`, donde se confirmó la presencia de las subcarpetas `browser` y `prerendered-routes.json`. Tras corregir la ruta pública en `firebase.json` y ejecutar nuevamente `firebase deploy`, la aplicación cargó correctamente.

8.7 Despliegue exitoso

Con la configuración corregida, el comando `firebase deploy` completó la subida de los archivos en pocos segundos y emitió el mensaje de éxito junto con la URL pública del proyecto:

```
Deploy complete!

Project Console: https://console.firebase.google.com/project/voto-universitario-770b7/overview
Hosting URL: https://voto-universitario-770b7.web.app
```

Desde ese momento, cualquier persona con un navegador y la extensión MetaMask configurada

en la red Sepolia puede acceder a la URL pública e interactuar con el sistema de votación. La separación entre frontend (Firebase) y backend (Sepolia) demuestra una arquitectura Web3 funcional en producción.

9 Cumplimiento del Documento de Visión

La siguiente tabla auditea el grado de cumplimiento de cada requisito especificado en el Documento de Visión original (fechado 11/05/2026).

Requisito del Documento de Visión	Estado	Implementación
Registrar candidatos participantes	Cumplido	Función <code>registrarCandidato(bytes32)</code> en Solidity. Panel admin en frontend.
Registrar votantes autorizados	Cumplido	Mapping <code>votanteAutorizado</code> , función <code>registrarVotante(address)</code> y <code>registrarVotantesEnLote</code> para escala masiva.
Abrir y cerrar procesos electorales	Cumplido	Funciones <code>abrirEleccion</code> y <code>cerrarEleccion</code> con eventos emitidos. Botones dedicados en UI.
Emitir votos mediante MetaMask	Cumplido	Firma criptográfica de cada voto vía ethers.js + MetaMask provider.
Garantizar un solo voto por usuario	Cumplido	Mapping <code>yaVoto[address]</code> consultado en <code>require</code> antes de aceptar la transacción.
Visualizar resultados en tiempo real	Cumplido	Función <code>obtenerTodosLosCandidatos()</code> consultada tras cada acción exitosa.
Registrar todas las transacciones en blockchain	Cumplido	Todas las operaciones de escritura son transacciones nativas Ethereum verificables en Etherscan.
Seguridad (RNF 6.1)	Cumplido	Tres modifiers de control de acceso + validaciones <code>require</code> + firma asimétrica MetaMask.
Disponibilidad (RNF 6.2)	Cumplido	Contrato vivo en Sepolia 24/7, sin dependencia de servidor centralizado.
Usabilidad (RNF 6.3)	Cumplido	Interfaz Angular con paneles claros, notificaciones temporales y banners de estado.
Escalabilidad (RNF 6.4)	Cumplido	Optimizaciones <code>bytes32</code> y <code>uint32</code> , función batch en contrato. Stack moderno actualizable.

10 Trabajo Futuro

A continuación se documentan las mejoras y extensiones identificadas durante el desarrollo, organizadas por nivel de madurez tecnológica. Estas representan la evolución natural del sistema más allá del alcance académico.

10.1 Carga masiva del padrón electoral mediante CSV

La función `registrarVotantesEnLote(address[])` ya implementada en el contrato permite cargar múltiples votantes en una sola transacción. En una segunda fase del proyecto, se desarrollaría un módulo en el frontend que permita al administrador cargar un archivo CSV con columnas matrícula, nombre y dirección Ethereum, y procesarlo en lotes de 200-500 direcciones para gestionar padrones de miles de estudiantes.

10.2 Auto-registro vinculado a identidad universitaria

Una evolución posterior contemplaría integrar el sistema con el SIIAU o SISI de la institución. Cada estudiante autenticado en el sistema universitario podría vincular su dirección de MetaMask con su matrícula, y al cerrar el periodo de inscripción, el padrón se cargaría automáticamente al contrato. Esto resolvería el problema de oráculo (vincular identidad real con identidad blockchain) de manera escalable.

10.3 Soulbound Tokens (SBT) como credencial estudiantil

Implementar un contrato adicional que emita NFTs no transferibles a cada estudiante activo. El contrato de votación verificaría la posesión del SBT en lugar de consultar una whitelist, permitiendo reutilizar la credencial digital en múltiples elecciones sin recargar padrones.

10.4 Privacidad mediante Zero-Knowledge Proofs

Para casos donde se requiera voto secreto verificable, se podrían integrar ZK-SNARKs que permitan probar criptográficamente que el votante es elegible y que solo votó una vez, sin revelar su identidad. Este enfoque es usado por proyectos como Vocdoni en España y permite combinar transparencia del proceso con anonimato del votante.

10.5 Despliegue en mainnet o L2

Para uso productivo, el contrato podría migrarse a una red de capa 2 como Polygon, Arbitrum u Optimism, donde los costos de gas son significativamente menores que en mainnet Ethereum, manteniendo la seguridad heredada del ecosistema Ethereum.

11 Conclusiones

El proyecto **Voto Universitario** logró implementar exitosamente todos los requisitos funcionales y no funcionales establecidos en el Documento de Visión, validando empíricamente el potencial de la tecnología blockchain para procesos electorales universitarios.

Durante el desarrollo se enfrentaron retos técnicos no triviales: la sincronización entre el provider Web3 y el frontend, la detección de cambios en Angular con código asíncrono externo, y el diseño de una arquitectura de seguridad robusta sin sacrificar la experiencia de usuario. Cada uno de estos retos fue documentado, diagnosticado y resuelto mediante soluciones técnicas justificadas.

La implementación del padrón electoral con whitelist on-chain es una contribución particular del proyecto que va más allá del minimum viable product: no solo cumple el requisito documental de *registrar votantes autorizados*, sino que demuestra dominio del control de acceso descentralizado y prepara el sistema para escalar a contextos productivos reales.

Desde el punto de vista académico, el proyecto sirvió como ejercicio práctico de los conceptos centrales de la materia: contratos inteligentes, máquinas de estado descentralizadas, firma criptográfica, eventos on-chain, optimización de gas, y arquitecturas DApp. Adicionalmente, el contacto directo con problemas reales del ecosistema Web3 (race conditions en confirmaciones, gestión de wallets, redes de prueba) proporciona experiencia transferible a entornos profesionales.

El sistema queda funcionando en la red Sepolia y puede ser auditado por terceros consultando la dirección del contrato en Etherscan. Adicionalmente, la interfaz frontend se encuentra publicada en <https://voto-universitario-770b7.web.app> a través de Firebase Hosting, accesible públicamente desde cualquier navegador con MetaMask configurado en la red Sepolia. Esta verificabilidad pública y accesibilidad universal son, en sí mismas, la prueba última de que los principios de transparencia, trazabilidad y descentralización enunciados en el Documento de Visión fueron honrados en la implementación.